

Realizzazione Server IRC

Programmazione modulare, I/O Multiplexing e multithreading

Siracusa Giacomo

Università degli Studi di Messina

Corso di Informatica

Laboratorio di Reti e Sistemi Distribuiti 2024-2025

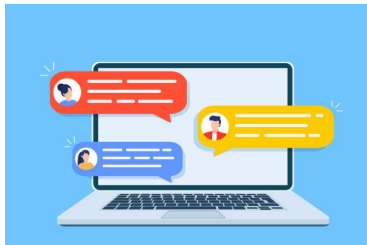
2 settembre 2025

- Creazione di un singolo Server IRC (Internet Relay Chat)
- Accesso al server in maniera concorrente tramite `select()` e multithreading
- Dati condivisi consistenti, tramite mutex e variabili di condizione
- Programmazione modulare

Cos'è IRC?

Il protocollo progettato per consentire conferenze basate su testo.
Due termini chiave principali:

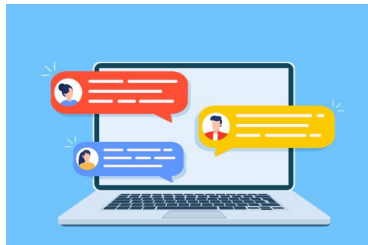
- Nickname
- Channel



Cos'è IRC?

Il protocollo progettato per consentire conferenze basate su testo.
Due termini chiave principali:

- Nickname
- Channel



Specifica: RFC 1459

Pubblicata nel 1993 da Jarkko Oikarinen e Darren Reed

Come funziona il protocollo?

- Applicato su sistemi con architettura di rete Client/Server e che utilizzano comunemente il protocollo di rete TCP/IP.
- Il server è un punto centrale, in cui i client possono connettersi e comunicare tramite comandi testuali.

Come funziona il protocollo?

- Applicato su sistemi con architettura di rete Client/Server e che utilizzano comunemente il protocollo di rete TCP/IP.
- Il server è un punto centrale, in cui i client possono connettersi e comunicare tramite comandi testuali.

Obiettivo principale del server: fare da tramite per le varie conferenze dei client.

Come funziona il protocollo?

- Applicato su sistemi con architettura di rete Client/Server e che utilizzano comunemente il protocollo di rete TCP/IP.
- Il server è un punto centrale, in cui i client possono connettersi e comunicare tramite comandi testuali.

Obiettivo principale del server: fare da tramite per le varie conferenze dei client.

Di seguito, il comando correlato

```
PRIVMSG #ricercaInformatica :Ciao!!
```

Altri comandi, ma non gli unici...

```
NICK, JOIN, TOPIC, PART, QUIT.
```

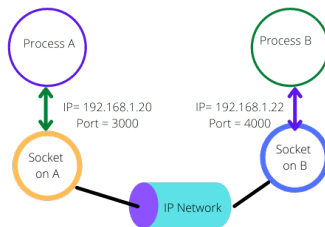
Sintassi generale dei comandi

```
<message> ::= <command> <params> <crlf>
<command> ::= <letter> { <letter> }
<SPACE>   ::= ' ' { ' ' }
<params>   ::= <SPACE> [ ':' <trailing> | <middle> <params> ]
<middle>   ::= <Qualsiasi sequenza di caratteri, escluso lo
               spazio, NUL, CR e LF, il primo carattere vietato ':'>
<trailing> ::= <Qualsiasi sequenza di caratteri, escluso NUL,
               CR e LF>
<crlf>     ::= CR LF
```

Messaggi di risposta: numerici a 3 cifre

Es: 461, significa che i parametri del comando sono insufficienti.

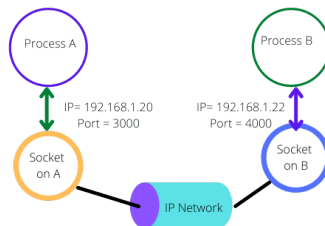
Grazie all'API Socket Linux (libreria in C `socket.h`):



Sistema Client-Server

Grazie all'API Socket Linux (libreria in C `socket.h`):

- Si può creare un Server TCP con le seguenti funzioni:
 - 1 `socket()`
 - 2 `binding()`
 - 3 `listen()`
 - 4 `accept()`
- Al Client invece ne bastano due:
 - 1 `socket()`
 - 2 `connect()`



Instaurata la connessione

I messaggi possono essere ricevuti e spediti tramite: `read()` e `write()`.

select()

Metodologia che implementa l'I/O Multiplexing.

Un insieme di socket viene monitorato in kernel space, appena ci sono operazioni di I/O o scade il limite di tempo d'attesa, il processo riparte e saprà su quali socket lavorare.

select()

Metodologia che implementa l'I/O Multiplexing.

Un insieme di socket viene monitorato in kernel space, appena ci sono operazioni di I/O o scade il limite di tempo d'attesa, il processo riparte e saprà su quali socket lavorare.

Multipiattaforma

La select() è compatibile anche su altri sistemi operativi

Costo computazionale

$\Theta(n)$, dove n è il numero di socket

Innanzitutto cos'è un thread?

Unità d'esecuzione base della CPU. Lo scheduler gestisce i thread e non i processi.

- Si ha quando un thread crea dei nuovi thread, aventi risorse condivise ma esecuzioni indipendenti.
- Se usato bene è uno strumento davvero potente e funzionale.

Innanzitutto cos'è un thread?

Unità d'esecuzione base della CPU. Lo scheduler gestisce i thread e non i processi.

- Si ha quando un thread crea dei nuovi thread, aventi risorse condivise ma esecuzioni indipendenti.
- Se usato bene è uno strumento davvero potente e funzionale.
- Libreria che offre il multithreading è pthread.h.

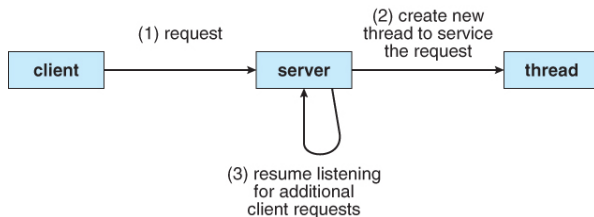


Figura: Es. di server multithread

Mutex e Variabili di condizione

Meccanismi di sincronizzazione che stanno bene insieme.

Mutex, implementa il concetto del lucchetto

Garantisce mutua esclusione per uno specifico blocco di istruzioni, chiamato anche sezione critica.

Variabile di condizione

Permettono di gestire l'esecuzione dei thread in base al valore dei loro dati condivisi.

- Da N ELF rilocabili correlati tra loro si ottiene un singolo file eseguibile.
- Si evitano così codici sorgenti con centinaia o migliaia di righe.
- Il compiler driver gcc riesce a fare ciò.

Dichiarazione variabile esterna

```
extern int a; //variabile globale definita su un altro file
```


Tipi di dati strutturati - users e channels

```
//service_server.h
typedef struct {
    int sockfd;
    char nickname[SIZE_NICKNAME];
}user;
typedef struct {
    user *array;
    int size;
}users;
typedef struct {
    char name[SIZE_CHANNEL_NAME];
    int fd_operator;
    int *fd_members;
    int n_members;
    char topic[SIZE_TOPIC];
}channel;
typedef struct {
    channel *array;
    int size;
}channels;
```

File principali che compongono il server

- 1 server.c, definisce il main() e start_IRC_protocol()
- 2 service_server.c, esegue i comandi IRC: nick, join, privmsg, ecc
- 3 utils_server.c ed utils_service_server.c che contengono funzioni di utilità

Table of Contents

- 1 server.c, definisce il main() e start_IRC_protocol()
- 2 service_server.c, esegue i comandi IRC: nick, join, privmsg, ecc
- 3 utils_server.c ed utils_service_server.c che contengono funzioni di utilità

Dati globali e il tipo "message"

```
pthread_cond_t cond=PTHREAD_COND_INITIALIZER;  
pthread_mutex_t lock= PTHREAD_MUTEX_INITIALIZER;  
int contatore_threads;
```

Dati globali e il tipo "message"

```
pthread_cond_t cond=PTHREAD_COND_INITIALIZER;  
pthread_mutex_t lock= PTHREAD_MUTEX_INITIALIZER;  
int contatore_threads;
```

Ogni messaggio ricevuto viene memorizzato tramite malloc() e passato come argomento al nuovo thread.

struct message

```
{ int sockfd; char buffer[BUFFER_SIZE] }
```

main() - Esecuzione della select()

```
int main(){
    ... //Dichiarazioni e inizializzazioni (socket compreso)
    while (!turn_off) {
        init_fd_set(&read_fds, array_sockfd, size_array);
        tv.tv_sec= 15; tv.tv_usec = 0;
        int ready_sockfd = select(max_sockfd+1, &read_fds,
NULL,NULL, &tv);
        ... // Controllo eventuale errore
        if (FD_ISSET(array_sockfd[0], &read_fds) &&
size_array < MAX_SOCKET) {
            ...//Accetto la nuova connessione
        }
        if (size_array>1) {
            int i=1, j=0;
            while (i<size_array && j<ready_sockfd && !
turn_off) {
                if(FD_ISSET(array_sockfd[i],&read_fds)){
                    //Leggo dal socket e creo il thread
                    ...
                }
            }
        }
    }
}
```

main() - Creazione thread

```
message *m=malloc(sizeof(message));
int n_bytes=read(array_sockfd[i],m->buffer,BUFFER_SIZE-1);
if (n_bytes >0) {
    m->buffer[n_bytes]='\0';
    ...//Controllo se si deve chiudere il server
    m->sockfd = array_sockfd[i];
    // attendo se ci sono MAX_NUM threads in esecuzione
    pthread_mutex_lock(&lock);
    contatore_threads++;
    while(contatore_threads>=MAX_NUM) {
        pthread_cond_wait(&cond, &lock);
    }
    pthread_mutex_unlock(&lock);

    if (!strncmp(m->buffer, "quit",4))
        close_connection_by_select(array_sockfd,&size_array
,&i);
    //Creazione thread!
    pthread_create(&thread, NULL, start_IRC_protocol, m);
    ...
}
```

start_IRC_protocol() - 1

```
void *start_IRC_protocol(void *arg) {
    message *m = (message *)arg;
    char *parameters=strchr(m->buffer, ' ');

    printf("FD: %d. Messaggio ricevuto: %s\n",m->sockfd,m->
buffer);
    if(parameters!=NULL){
        parameters[0]='\0';
        parameters++;
        if (parameters[strlen(parameters)-2]=='\r' &&
parameters[strlen(parameters)-1]=='\n') {
            parameters[strlen(parameters)-2]='\0';
            if (!strcmp(m->buffer, "nick"))
                nick(m->sockfd, parameters);
            else if (!strcmp(m->buffer, "join"))
                join(m->sockfd, parameters);
            else if (!strcmp(m->buffer, "part"))
                part(m->sockfd, parameters);
            else if ... //topic, quit e privmsg
        }
    }
    ...
}
```


start_IRC_protocol() - 2

```
    }else
        send_response(m->sockfd, "421 Comando
sconosciuto");
}else {
    if (!strncmp(m->buffer, "quit",4))
        quit(m->sockfd);
    else
        send_response(m->sockfd, "421 Comando
sconosciuto");
}

free(m); //deallocazione del messaggio dall'heap

pthread_mutex_lock(&lock);
contatore_threads--;
pthread_cond_signal(&cond);
pthread_mutex_unlock(&lock);

return NULL;
}
```

Table of Contents

- 1 server.c, definisce il main() e start_IRC_protocol()
- 2 service_server.c, esegue i comandi IRC: nick, join, privmsg, ecc
- 3 utils_server.c ed utils_service_server.c che contengono funzioni di utilità

nick() - Prima parte

```
void nick(int sockfd, char* nickname) {
    char buffer[BUFFER_SIZE-2]={0};
    if (strlen(nickname)>0) {
        ...//riduco la lunghezza se e' maggiore di SIZE_NICK
        if (is_middle(nickname) && nickname[0]!='#') {
            pthread_mutex_lock(&mutex);
            user *p_u=get_user(sockfd);
            if (p_u!=NULL) {//se utente trovato
                if (get_user_by_nickname(nickname)!=NULL){
                    if (!strcmp(p_u->nickname, nickname))
                        sprintf(buffer, "433 %s :Nickname
gia' in uso", nickname);
                    else
                        sprintf(buffer, "436 %s :Collisione
di nickname", nickname);

                    send_response(sockfd, buffer);
                }else // Imposto il nuovo nick
                    strcpy(p_u->nickname, nickname);
            }
        }
    }
}
```

nick() - Seconda parte

```
    }else { //creo un nuovo utente
        if (++u.size==1)
            u.array=malloc(sizeof(user));
        else
            u.array=realloc(u.array,u.size*sizeof(
user));

        p_u=&u.array[u.size-1];
        p_u->sockfd=sockfd;
        strcpy(p_u->nickname, nickname);
        printf("Utente creato: %s\n",p_u->nickname);
    }
    pthread_mutex_unlock(&mutex);
} else {
    sprintf(buffer, "432 %s :Nickname non valido",
nickname);
    send_response(sockfd, buffer);
}
} else
    send_response(sockfd, "432 nick :Nickname non dato");
}
```

Table of Contents

- 1 server.c, definisce il main() e start_IRC_protocol()
- 2 service_server.c, esegue i comandi IRC: nick, join, privmsg, ecc
- 3 utils_server.c ed utils_service_server.c che contengono funzioni di utilità

Tutte le funzioni chiamate da service_server.c

```
//service_server.h:
... MACRO e strutture dati

//utils_server.c
void send_response(int sockfd, char response[]);

//utils_service_server.c
bool is_middle(char *parameter);
bool is_trailing(char * parameter);
user *get_user(int sockfd);
user *get_user_by_nickname(char *nickname);
channel *get_channel(char *c_name);
void join_in_a_channel(channel *p_ch, user *p_u, int sockfd);
bool is_member(channel *p_ch, int sockfd);
char *get_channel_name(char *dest, char *source);
bool exit_the_channel(int sockfd, channel *p_ch);
void send_to_client(char *nickname, user *p_u, char *text);
void send_to_channel(char *ch_name, user *p_u, char *text);
```

```
$ ./client
Connessione al server attiva. Invia messaggi (scrivi 'help'
per la guida):
nick Chiara
join #serietv
Sat Aug 30 23:15:56 2025: 353 #serietv :Chiara
Sat Aug 30 23:15:56 2025: 331 #serietv :Nessun argomento
presente
Sat Aug 30 23:16:33 2025: 301 #serietv - Aurelio :Ciao :)
Aurelio :Che serie tv mi consigli?
```

L'ultimo messaggio inviato è privato, e solo Aurelio lo riceverà.

```
nick Giacomo
join #sport,#serietv
Sat Aug 30 23:17:06 2025: 353 #sport :Giacomo
331 #sport :Nessun argomento presente
353 #serietv :Chiara,Aurelio,Giacomo
331 #serietv :Nessun argomento presente
topic #sport :Notizie sportive
Sat Aug 30 23:17:39 2025: 332 #sport :Notizie sportive
#sport :Ma quanto e' forte Sinner?
Sat Aug 30 23:18:57 2025: 301 #sport - Laura :Tropo!
part #sport,#serietv
```


Realizzare questo progetto, oltre ad essere stato utile, mi ha fatto comprendere la logica di base che sta dietro ai sistemi di messaggistica che utilizzo quotidianamente. Il materiale mostrato è solo una base di partenza, su cui in futuro, possono essere aggiunte tantissime funzionalità.